ARCHITECTURE REVIEW

Honest Critique of the Previous AI Roadmap

A shareable review of the earlier “Personal Expense & Splitwise Sync Bridge” plan: what is strong, what is missing, what is overcomplicated, and what should be changed before implementation.

7.5/10
Overall score

Good
Core concept

Weak
Settlement model

Early
Plaid + AI voice

1. Overall Verdict

> **Verdict:** The previous roadmap is directionally strong, but it is not the best final architecture. It correctly identifies the difference between card charges and personal expenses, but it under-models reimbursements, payables, settlements, and real-world ledger states.

The strongest part is the idea that credit card transactions should enter an inbox first and only your actual personal share should count toward the $800 budget. That should be kept.

The weakest part is the database model. It is too simple for the actual problem because your life includes cash repayments, partial settlements, people paying for you, offsets, and delayed splitting.

2. What the Previous AI Got Right

KEEP

It separated cash flow from personal spending

This is the correct foundation. A $100 grocery transaction on your card is not necessarily a $100 personal expense. Only your allocated share should count toward the monthly budget.

KEEP

It used an unprocessed transaction inbox

This respects the real constraint that you cannot split everything immediately. Imported transactions should be allowed to sit unresolved until you are free.

KEEP

It started with CSV import

Starting with CSV is practical. It avoids bank-linking, webhooks, token management, and financial data security during MVP.

KEEP BUT REFINE

It proposed Splitwise integration

Splitwise integration is useful, but it should be treated as a sync/communication layer. The local app should remain the source of truth for the budget.

3. Major Flaws and Gaps

CRITICAL

Flaw 1: No proper settlements table

The previous schema tracks transactions and splits, but it does not properly model repayments. This is a major issue because reimbursements are central to the use case.

Why it matters: If Logan owes $80, pays $50 cash, and later buys something worth $30 for you, the app needs to represent that cleanly. Without settlements, balances become unreliable.

Fix: Add a first-class settlements table.

```
settlements
- id
- settlement_date
- from_person_id
- to_person_id
- amount
- currency
- method: cash / e-transfer / card / offset / splitwise
- notes
```

CRITICAL

Flaw 2: It does not fully handle “someone else bought something for me”

The roadmap focuses mostly on transactions that appear on your own credit card. But your problem also includes expenses that belong to you but were paid by someone else.

Why it matters: If Rahul pays for your $18 dinner share, your card shows nothing, but your $800 budget should still increase by $18 and you should owe Rahul $18.

Fix: Support manual transactions with source type `manual_someone_paid_for_me` or create payables directly through allocations.

CRITICAL

Flaw 3: It models receivables more clearly than payables

The plan explains "Housemates Owe Me," but it does not equally model "I Owe Someone Else." Real balances need both directions.

Fix: Every allocation should be able to create either a receivable or payable depending on who paid and who consumed.

MAJOR

Flaw 4: is_processed is too simple

A boolean state cannot capture the lifecycle of a transaction. Transactions can be unreviewed, partially allocated, allocated, ignored, refunded, duplicate, sync-failed, or reconciled.

Fix: Use a richer status field.

```
status:
- imported
- needs_review
- partially_allocated
- allocated
- ignored
- refunded
- duplicate
```

```
  - sync_failed
  - reconciled
```

MAJOR

Flaw 5: Person modeling uses strings instead of real entities

The previous schema uses assigned_to_user VARCHAR. That is fine for a tiny prototype but weak for a real app.

Why it matters: Names can change, people can belong to groups, and Splitwise user IDs need stable mapping.

Fix: Add people, groups, and group_members tables. Allocations should reference person_id, not a display name string.

MAJOR

Flaw 6: Splitwise sync is under-designed

Storing a Splitwise expense ID is useful, but not enough. Real sync needs retry handling, errors, timestamps, idempotency, and a queue of failed sync events.

Fix: Add sync_events with status, error message, provider, external ID, retry count, and timestamps.

MAJOR

Flaw 7: Plaid is introduced too early

Bank sync should not be part of the early build. It adds complexity before the product logic is validated.

Fix: Make CSV import the primary ingestion method for MVP. Add Plaid/open banking only after the ledger, inbox, allocation, and settlement flows are useful.

MAJOR

Flaw 8: AI voice is introduced before the core workflow is proven

Voice input is a great future feature, but it should not be part of the core foundation.

Fix: Build reliable manual flows first. Then add AI voice commands that produce draft transactions, allocations, or settlements for confirmation.

MAJOR

Flaw 9: It over-splits frontend and backend too early

A separate React app plus Express backend is valid, but for a solo personal project it can create unnecessary deployment and maintenance overhead.

Fix: Consider a single Next.js app with API routes/server actions, Postgres, and a PWA frontend.

MAJOR

Flaw 10: Budget date semantics are not clearly defined

Budget calculations need a clear decision: should spending count by transaction date, allocation date, statement date, or processing date?

Recommended rule: Count spending by the actual expense date. Processing date should not decide the budget month unless intentionally chosen.

4. Overcomplicated Parts

AREA	PROBLEM	RECOMMENDATION

Plaid in early roadmap	Too much financial-data complexity before MVP validation.	Delay until CSV workflow is proven.
AI voice early	Cool feature, but not required to prove the ledger.	Build after mobile-friendly manual flows.
Separate Express backend	Valid, but more moving parts for one developer.	Use Next.js full-stack unless there is a strong reason not to.
Immediate Splitwise automation	Sync bugs could pollute social balances.	Start with local balances, then add controlled sync with review/retry.

5. Missing Concepts That Should Be Added

- **Settlements:** cash, e-transfer, Splitwise settlement, and offset repayments.
- **Payables:** money you owe others when they paid for your consumption.
- **People and groups:** stable local identity mapping instead of plain text names.
- **Manual expense source:** for expenses not appearing on your credit card.
- **Sync event log:** Splitwise retry/error handling.
- **Import history:** audit CSV uploads and duplicate handling.
- **Budget periods:** avoid hardcoding $800 forever.
- **Transaction statuses:** richer lifecycle than processed/unprocessed.
- **Refund handling:** negative allocations or linked refund transactions.
- **Audit trail:** useful because money data often needs correction.

6. Better Core Model

> **Recommended replacement model:** Transactions are payment events. Allocations are consumption ownership. Settlements clear debts. Budget is derived from personal allocations only.

```
transactions
  Raw payment/manual events.

allocations
  Who consumed which amount.
  Determines budget impact and receivable/payable creation.

settlements
  Money or value transferred to clear balances.

people
  Stable local identities mapped to Splitwise users later.

sync_events
  External API operations, retries, failures, and external IDs.
```

7. Suggested Message Back to the Other AI

Your roadmap is a strong starting point, especially the separation between card cash flow and personal expense, and the unprocessed transaction inbox. However, the architecture is incomplete for the real-world reimbursement problem. The biggest gaps are the lack of a first-class settlements table, no clean model for payables when someone else pays for me, a too-simple is_processed boolean, string-based person modeling, and insufficient Splitwise sync reliability. I would redesign the core around transactions, allocations, settlements, people, and sync events. I would also delay

Plaid and AI voice until the local CSV/import/allocation/settlement workflow is proven.

8. Final Scorecard

CATEGORY	SCORE	COMMENT
Problem understanding	9/10	Correctly understands that card charges are not equal to personal spending.
MVP direction	8/10	CSV first and inbox review are good decisions.
Database design	5.5/10	Too simple for reimbursements, payables, settlements, and identity mapping.
Integration design	6.5/10	Splitwise is useful but sync reliability needs more detail.
Roadmap sequencing	6/10	Plaid and AI are introduced too early.
Overall	7.5/10	Good concept, but core ledger architecture needs refinement before implementation.